

POLITECHNIKA ŁÓDZKA

WYDZIAŁ FIZYKI TECHNICZNEJ, INFORMATYKI I
MATEMATYKI STOSOWANEJ

Optymalna transversala macierzy (algorytm węgierski)

Autor:

Iga Stachowiak

Nr albumu: 263321

Przedmiot:

Algorytmy Optimalizacji

Dyskretnej

Spis treści

1	Wprowadzenie do problemu	2
2	Praktyczne zastosowania	2
3	Rozwiązanie problemu krok po kroku	2
4	Implementacja w języku Python	3
5	Rozszerzenie problemu: Maksymalizacja zysków	4
5.1	Analiza działania transformacji	4
6	Podsumowanie i wnioski	5

1 Wprowadzenie do problemu

Problem wyboru **optymalnej transwersali macierzy**, znany szerzej w literaturze jako **problem przydziału** (ang. *assignment problem*) lub problem skojarzenia o minimalnym koszcie w grafie dwudzielnym ważonym, jest jednym z klasycznych zagadnień optymalizacji kombinatorycznej.

W ujęciu formalnym dysponujemy kwadratową macierzą kosztów C o wymiarach $n \times n$, gdzie element $c_{i,j}$ reprezentuje koszt przypisania i -tego zasobu (np. pracownika, maszyny) do j -tego zadania. Poszukujemy takiej permutacji σ zbioru $\{1, \dots, n\}$, która zminimalizuje globalną funkcję celu, określoną jako suma kosztów wybranych elementów macierzy:

$$\min_{\sigma} \sum_{i=1}^n c_{i,\sigma(i)} \quad (1)$$

Definicja transwersali nakłada restrykcyjne ograniczenie: z każdego wiersza i z każdej kolumny macierzy możemy wybrać dokładnie jeden element. Algorytm węgierski (metoda Kuhna-Munkresa) pozwala na znalezienie tego optimum w czasie wielomianowym $\mathcal{O}(n^3)$, co stanowi kluczowe usprawnienie względem algorytmu przeglądu zupełnego o złożoności $\mathcal{O}(n!)$.

2 Praktyczne zastosowania

Algorytm węgierski stanowi fundament teoretyczny oraz narzędziowy w wielu obszarach inżynierii i nauk o zarządzaniu:

- **Badania operacyjne i logistyka:** Optymalne kierowanie pojazdów kurierskich, przydział zleceń transportowych do przewoźników.
- **Zarządzanie zasobami ludzkimi:** Harmonogramowanie pracy personelu, gdzie pracownicy posiadają różne stawki oraz stopnie efektywności dla poszczególnych zadań.
- **Automatyzacja produkcji:** Optymalizacja sekwencji operacji maszynowych w celu minimalizacji czasu przestojów i kosztów przebrojeń.
- **Wizja komputerowa:** Kojarzenie punktów kluczowych lub śledzenie wielu obiektów jednocześnie (ang. *multi-object tracking*) pomiędzy kolejnymi klatkami sekwencji wideo.

3 Rozwiązanie problemu krok po kroku

Algorytm opiera się na twierdzeniu Kőniga-Egerváry’ego, z którego wynika, że odjęcie lub dodanie stałej wartości od elementów dowolnego wiersza bądź kolumny nie zmienia struktury rozwiązania optymalnego, a modyfikuje jedynie wartość funkcji celu. Standardowa procedura składa się z następujących kroków:

1. **Redukcja wierszy:** W każdym wierszu macierzy kosztów identyfikowana jest wartość minimalna, która następnie zostaje odjęta od wszystkich elementów tego wiersza.

2. **Redukcja kolumn:** W każdej kolumnie nowo powstałej macierzy identyfikowana jest wartość minimalna, która zostaje odjęta od wszystkich elementów danej kolumny.
3. **Test optymalności (Wykreślanie zer):** Wyznacza się minimalną liczbę linii (poziomych i pionowych) niezbędnych do przykrycia wszystkich zer w macierzy. Jeśli liczba linii jest równa n (rozmiarowi macierzy), znaleziono rozwiązanie optymalne i proces się kończy. W przeciwnym razie następuje przejście do kroku 4.
4. **Korekta macierzy:** Spośród wszystkich elementów nieprzykrytych żadną linią wybiera się wartość najmniejszą (k). Wartość k jest odejmowana od każdego niezakrytego elementu oraz dodawana do elementów znajdujących się na przecięciach linii pokrywających. Po wykonaniu korekty następuje powrót do kroku 3.

4 Implementacja w języku Python

Do celów wdrożeniowych wykorzystano bibliotekę SciPy, która dostarcza zoptymalizowaną funkcję `linear_sum_assignment`. Poniższy skrypt prezentuje rozwiązanie przykładowego problemu przydziału o wymiarach 3×3 .

```

1 import numpy as np
2 from scipy.optimize import linear_sum_assignment
3
4 def oblicz_optymalna_transwersale():
5     # Definicja macierzy koszt w (Pracownicy x Zadania)
6     cost_matrix = np.array([
7         [10, 15, 9],
8         [9, 18, 5],
9         [6, 14, 3]
10    ])
11
12    print("Macierz koszt w wej ciowych:")
13    print(cost_matrix)
14    print("-" * 50)
15
16    # Wyznaczenie optymalnego przydzia u
17    row_ind, col_ind = linear_sum_assignment(cost_matrix)
18
19    # Wy wietlenie wynik w
20    print("Optymalny przydzia (indeksy wiersz -> kolumna):")
21    for r, c in zip(row_ind, col_ind):
22        print(f" Pracownik {r} ---> Zadanie {c} (Koszt: {cost_matrix[r, c]} PLN)")
23
24    total_cost = cost_matrix[row_ind, col_ind].sum()
25    print("-" * 50)
26    print(f"Suma minimalna (koszt ca kowity): {total_cost} PLN")
27
28 if __name__ == "__main__":
29     oblicz_optymalna_transwersale()

```

Listing 1: Implementacja algorytmu węgierskiego w Pythonie

5 Rozszerzenie problemu: Maksymalizacja zysków

Domyślnie algorytm węgierski (oraz funkcja `linear_sum_assignment`) służy do minimalizacji kosztów. W praktyce biznesowej często pojawia się jednak potrzeba **maksymalizacji zysków, wydajności lub przychodów**.

Aby zaadaptować algorytm do problemu maksymalizacji, należy dokonać matematycznego przekształcenia macierzy wejściowej. Najprostszą i najbardziej elegancką metodą jest pomnożenie wszystkich elementów macierzy zysków P przez -1 :

$$C = -P \quad (2)$$

W ten sposób najwyższe zyski stają się „najmniejszymi kosztami” (najbardziej ujemnymi wartościami), co pozwala funkcji minimalizującej na prawidłowe wskazanie optymalnego przypisania.

Poniższy kod prezentuje praktyczną implementację tego podejścia dla trzech pracowników i trzech projektów o różnych estymowanych zyskach.

```
1 import numpy as np
2 from scipy.optimize import linear_sum_assignment
3
4 def maksymalizacja_zysku():
5     # Definicja macierzy zysk w (Pracownicy x Projekty)
6     profit_matrix = np.array([
7         [40, 70, 55],
8         [45, 60, 50],
9         [30, 80, 65]
10    ])
11
12    # Przekształcenie: maksymalizacja -> minimalizacja
13    cost_matrix = profit_matrix * -1
14
15    # Uruchomienie algorytmu na zmodyfikowanej macierzy
16    row_ind, col_ind = linear_sum_assignment(cost_matrix)
17
18    # Prezentacja wynik w z u yciem oryginalnych wartości zysk w
19    print("Optymalny przydział dla maksymalizacji zysku:")
20    for r, c in zip(row_ind, col_ind):
21        zysk = profit_matrix[r, c]
22        print(f"Pracownik {r} ---> Projekt {c} | Zysk: {zysk} tys. zł")
23
24    max_total_profit = profit_matrix[row_ind, col_ind].sum()
25    print("-" * 50)
26    print(f"Maksymalny zysk całkowity: {max_total_profit} tys. zł")
27
28 if __name__ == "__main__":
29     maksymalizacja_zysku()
```

Listing 2: Implementacja maksymalizacji zysku przy użyciu algorytmu węgierskiego

5.1 Analiza działania transformacji

Dla powyższej macierzy, bez transformacji, algorytm szukałby najmniejszych liczb. Po przemnożeniu przez -1 , wartość 80 (maksymalny zysk w macierzy) zamienia się w -80 . W procesie minimalizacji wartość -80 jest traktowana jako najmniejsza możliwa (najbardziej atrakcyjna), przez co algorytm nada jej najwyższy priorytet podczas

wyboru transversali. Przy odczytywaniu wyniku końcowego sumujemy oczywiście wartości z pierwotnej macierzy P , otrzymując poprawny, maksymalny zysk.

6 Podsumowanie i wnioski

Algorytm węgierski jest potężnym narzędziem matematycznym pozwalającym unikać problemów związanych z eksplozją kombinatoryczną w zagadnieniach dyskretnych. Dzięki reprezentacji macierzowej i prostym operacjom odejmowania, algorytm zbiega do wyniku optymalnego w sposób deterministyczny. Współczesne implementacje biblioteczne (takie jak zaprezentowany moduł języka Python) pozwalają na błyskawiczne rozwiązywanie problemów przydziału dla struktur rzędu tysięcy zmiennych.